



SQL Injection

- 
- > ¿Qué es SQL?
 - > SQL Injection
 - > Ataques UNION
 - > Examinando la base de datos
 - > Blind SQLi
 - > Contramedidas para SQLi
 - > Herramientas



> ¿Qué es SQL?

- > SQL Injection
- > Ataques UNION
- > Examinando la base de datos
- > Blind SQLi
- > Contramedidas para SQLi
- > Herramientas

Lenguaje SQL

- > SQL o Structured Query Language es un lenguaje diseñado para administrar, y recuperar información de bases de datos relacionales .
- > Basado en álgebra relacional y en el cálculo relacional, SQL consiste en:
 - Un lenguaje de definición de datos , que nos permite crear y modificar esquemas. (CREATE, ALTER, DROP)
 - Un lenguaje de manipulación de datos , que nos permite: insertar, consultar, actualizar y borrar datos. (SELECT, INSERT, UPDATE, DELETE)
 - Un lenguaje de control de datos con el cual controlamos el acceso a los datos. (GRANT, REVOKE)

Cómo funciona?



> ¿Qué es SQL?

> SQL Injection

> Ataques UNION

> Examinando la base de datos

> Blind SQLi

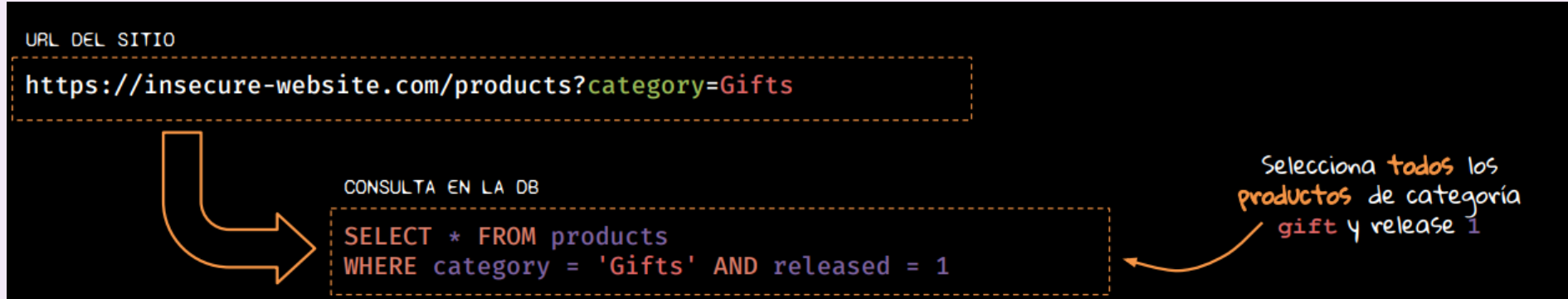
> Contramedidas para SQLi

> Herramientas

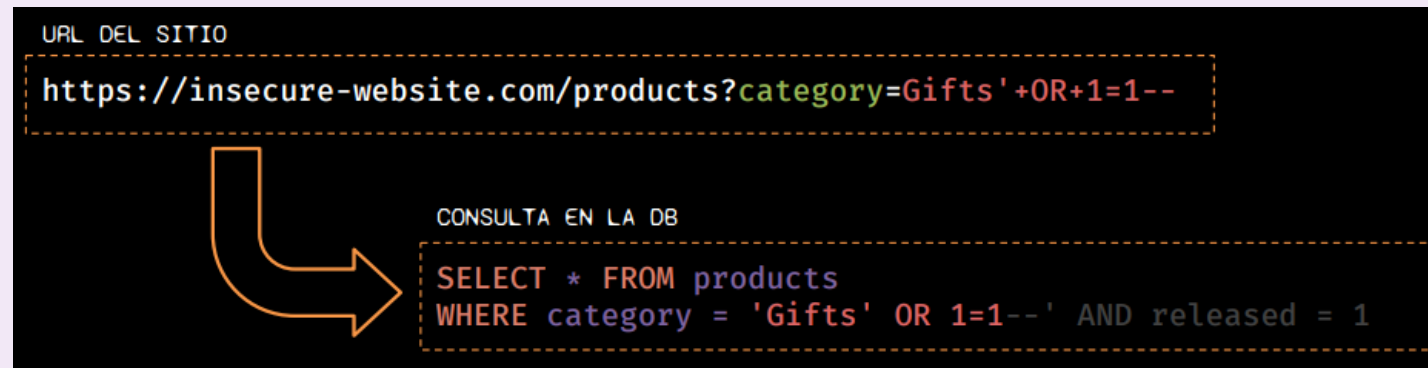
Qué es SQL Injection? (SQLi)

- > La inyección de SQL es una vulnerabilidad de seguridad web que permite a un atacante interferir con las consultas que una aplicación realiza a su base de datos.
- > Permite que un atacante vea datos que normalmente no puede recuperar, incluyendo:
 - datos pertenecientes a otros usuarios
 - cualquier otro dato al que la propia aplicación pueda acceder.
- > En muchos casos, un atacante puede modificar o eliminar estos datos, provocando cambios persistentes en el contenido o el comportamiento de la aplicación.
- > En algunas situaciones, un atacante puede utilizar un ataque de inyección SQL para comprometer el servidor u otra infraestructura de back-end, o realizar un ataque de denegación de servicio.

SQLi en acción :)



SQLi en acción :)



SQLi: login bypass

```
POST /rest/user/login HTTP/1.1
Host: insecure-website.com
Content-Length: 32
sec-ch-ua:
Origin: http://www.insecure-website.com
Referer: http://www.insecure-website.com
Accept-Encoding: gzip, deflate
Accept-Language: es-419,es;q=0.9

{
  "email": "tungur",
  "password": "pa$$w0rD*"
}
```



```
SELECT * FROM users
WHERE username = 'tungur'
AND password = 'pa$$w0rD*'
```

Si la consulta devuelve un usuario
el inicio de sesión es válido.

```
POST /rest/user/login HTTP/1.1
Host: insecure-website.com
Content-Length: 32
sec-ch-ua:
Origin: http://www.insecure-website.com
Referer: http://www.insecure-website.com
Accept-Encoding: gzip, deflate
Accept-Language: es-419,es;q=0.9

{
  "email": "admin'--",
  "password": "nn"
}
```



```
SELECT * FROM users
WHERE username = 'admin'--'
AND password = 'pa$$w0rD*'
```

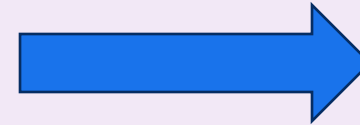
- > ¿Qué es SQL?
- > SQL Injection
- > Ataques UNION
- > Examinando la base de datos
- > Blind SQLi
- > Contramedidas para SQLi
- > Herramientas

Cómo funciona UNION

nombre	apellido
Ricardo	Bochini
Diego	Maradona
Lional	Messi

nombre	apellido
Novak	Djokovic
Roger	Federer
Rafael	Nadal

```
SELECT F.nombre, F.apellido FROM football as F  
UNION SELECT T.nombre, T.apellido FROM tennis as T
```



nombre	apellido
Ricardo	Bochini
Diego	Maradona
Lional	Messi
Novak	Djokovic
Roger	Federer
Rafael	Nadal

Para que UNION funcione se necesita:

- ☐ Mismo número de columnas
- ☐ Tipos de datos compatibles

Ataque UNION: número de columnas

Para hacer un ataque union tendremos que averiguar:

- (1) Cuántas columnas se devuelven en la consulta original?
- (2) Qué columnas devueltas de la consulta original son de un tipo de datos adecuado para contener los resultados de la consulta inyectada?

Método #1

CONSULTA EN LA DB

```
SELECT * FROM products  
WHERE category = '' ORDER BY 1 --' AND released = 1
```



CONSULTA EN LA DB

```
SELECT * FROM products  
WHERE category = '' ORDER BY 2 --' AND released = 1
```



CONSULTA EN LA DB

```
SELECT * FROM products  
WHERE category = '' ORDER BY 3 --' AND released = 1
```



El ORDER BY en la posición 3 se encuentra fuera de rango

Método #2

CONSULTA EN LA DB

```
SELECT * FROM products  
WHERE category = '' UNION SELECT NULL--' AND...
```



CONSULTA EN LA DB

```
SELECT * FROM products  
WHERE category = '' UNION SELECT NULL,NULL--' AND...
```



CONSULTA EN LA DB

```
SELECT * FROM products  
WHERE category = '' UNION SELECT NULL,NULL,NULL--' AND...
```



Todas las consultas combinadas mediante UNION, INTERSECT o EXCEPT debe tener el mismo número de expresiones en sus listas de objetivos.

- > ¿Qué es SQL?
- > SQL Injection
- > Ataques UNION
- > Examinando la base de datos
- > Blind SQLi
- > Contramedidas para SQLi
- > Herramientas

Intro

- > Al explotar las vulnerabilidades de SQLi generalmente es necesario recopilar información sobre la propia base de datos.
- > Esto incluye:
 - el tipo y la versión del software de la base de datos
 - y el contenido de la base de datos en términos de las tablas y columnas que contiene.



Conocer el tipo y versión

- > Las diferentes bases de datos proporcionan diferentes formas de consultar su versión.
- > Vamos a tener que probar diferentes consultas para encontrar una que funcione
- > Con esta técnica, probablemente podamos determinar tanto el tipo como la versión del software de la base de datos.
- > Las consultas para determinar la versión de la base de datos para algunos tipos de bases de datos más populares son las siguientes:

Tipo de Base de Datos	Consulta
SQL, MySQL	SELECT @@version
Oracle	SELECT * FROM v\$version
PostgreSQL	SELECT version()

Conocer el tipo y versión

- > Por ejemplo, podría usar un ataque UNION:
 - o ' UNION SELECT @@version--
- > Esto podría devolver un resultado como el siguiente, confirmando que la base de datos es Microsoft SQL Server y la versión que se está utilizando:

```
Microsoft SQL Server 2016 (SP2) (KB4052908) - 13.0.5026.0 (X64)
Mar 18 2018 09:11:49
Copyright (c) Microsoft Corporation
Standard Edition (64-bit) on Windows Server 2016 Standard 10.0 <X64> (Build 14393: ) (Hypervisor)
```

Listando el contenido de la Base de Datos

- > La mayoría de los tipos de bases de datos (con excepción de Oracle) tienen un conjunto de vistas llamado esquema de información que proporciona información sobre la base de datos.
- > Se puede consultar `information_schema.tables` para listar las tablas en la base de datos:

```
SELECT * FROM information_schema.tables
```

- > Esto devuelve un resultado como el siguiente, donde hay tres tablas, llamadas Products , Users y Feedback .

```
TABLE_CATALOG TABLE_SCHEMA TABLE_NAME TABLE_TYPE
=====
MyDatabase dbo Products BASE TABLE
MyDatabase dbo Users BASE TABLE
MyDatabase dbo Feedback BASE TABLE
```

Listando el contenido de la Base de Datos

> Luego puede consultar `information_schema.columns` para listar las columnas en tablas individuales:

```
SELECT * FROM information_schema.columns WHERE table_name = 'Users'
```

> Esto devuelve un resultado como el siguiente, con las columnas de la tabla especificada y el tipo de datos de cada columna

```
TABLE_CATALOG TABLE_SCHEMA TABLE_NAME COLUMN_NAME DATA_TYPE
=====
MyDatabase    dbo      Users    UserId    int
MyDatabase    dbo      Users    Username  varchar
MyDatabase    dbo      Users    Password  varchar
```

SQL INJECTION CHEAT SHEETS

Sitio	Link
Portswigger	https://portswigger.net/web-security/sql-injection/cheat-sheet
Hackthebox	https://www.hackthebox.com/files/cheatsheet-sql-injection-fundamentals.pdf
Github repo – advanced queries	https://github.com/kleiton0x00/Advanced-SQL-Injection-Cheatsheet



- > ¿Qué es SQL?
- > SQL Injection
- > Ataques UNION
- > Examinando la base de datos
- > Blind SQLi
- > Contramedidas para SQLi
- > Herramientas

Qué es Blind SQL?

- > Blind SQLi surge cuando una aplicación es vulnerable a la inyección SQL, pero sus respuestas HTTP no contienen los resultados de la consulta SQL relevante o los detalles de los errores de la base de datos.
- > Blind SQLi hace que muchas técnicas, como por ejemplo los ataques de tipo UNION, no sean efectivas .
- > Esto ocurre porque se basan en poder ver los resultados de la consulta inyectada dentro de las respuestas de la aplicación.
- > De todas formas, es posible explotar la vulnerabilidad blind SQLi para acceder a datos no autorizados, pero se deben utilizar técnicas diferentes.

Técnicas de Blind SQL

> Si bien existen diferentes técnicas para poder explotar Blind SQLi, veremos las siguientes:

- Con respuestas condicionales .
- Provocando errores condicionales .
- Provocando retrasos de tiempos .



Técnicas de Blind SQL – Respuestas condicionales

> Es una técnica de ataque en la que el atacante no ve los resultados directos de las consultas SQL inyectadas, sino que infiere información a través de diferencias en las respuestas del servidor, como cambios en el contenido renderizado, códigos HTTP o mensajes específicos

Estrategias clave:

- Validación booleana
- Extracción de datos caracter por caracter
- Automatización con scripts

Técnicas de Blind SQL – Respuestas condicionales: Validación booleana

- > Se envían consultas que devuelven una respuesta verdadera o falsa:
 - **AND 1=1** → respuesta normal (ej. mensaje "Welcome back!").
 - **AND 1=2** → respuesta diferente (ej. mensaje ausente).
- > Si el contenido cambia, se confirma la vulnerabilidad.

Técnicas de Blind SQL – Respuestas condicionales – Extracción de caracteres

> Se usa `SUBSTRING()` o `MID()` para comparar un carácter a la vez:

- `' AND SUBSTRING((SELECT password FROM users WHERE username='administrator'), 1, 1) = 'a'`

→ Si la respuesta es verdadera (por ejemplo, el mensaje "Welcome back!" aparece), se sabe que el primer carácter es 'a'.

Técnicas de Blind SQL – Respuestas condicionales – Automatización con scripts

- > Se utilizan herramientas como Python para probar todos los caracteres posibles de forma iterativa.
- > Un script típico:
 1. Itera sobre cada posición de la contraseña.
 2. Prueba cada carácter (letras y números).
 3. Verifica la presencia del mensaje esperado en la respuesta.

Técnicas de Blind SQL – Errores condicionales

- > Es una técnica avanzada donde el atacante induce un error en la base de datos solo cuando una condición inyectada es verdadera, permitiendo inferir información a partir de la presencia o ausencia de errores en la respuesta.
- > Mecanismo clave: Se utiliza una sentencia condicional (como **CASE** o **IF**) que genera un error (por ejemplo, división por cero $1/0$) si la condición es verdadera, pero no si es falsa. La diferencia en la respuesta (como un error HTTP 500) indica el resultado de la condición

```
' AND (SELECT CASE WHEN (1=1) THEN TO_CHAR(1/0) ELSE " END FROM dual) = "
```

- Si la condición es verdadera ($1=1$), se produce un error ([ORA-01476](#)).
- Si es falsa ($1=2$), no hay error

Técnicas de Blind SQL – Retrasos de tiempos (time based)

- > Técnica utilizada cuando la aplicación no devuelve respuestas visibles (como errores o cambios en el contenido), pero sí permite detectar diferencias en el tiempo de respuesta.
- > El atacante inyecta condiciones que provocan retrasos en la base de datos si se cumplen, permitiendo inferir información mediante la medición del tiempo de respuesta.

Técnicas de Blind SQL – Retrasos de tiempos (time based)

> Cómo funciona:

1. El atacante inyecta una consulta que fuerza al sistema a esperar (delay) si una condición es verdadera.
2. Si la respuesta tarda más de lo normal (por ejemplo, 5 segundos), se concluye que la condición es verdadera.
3. Este proceso se repite caracter a caracter para extraer información como versiones de base de datos, nombres de tablas o credenciales

```
' AND IF(SUBSTRING(@@version,1,1)='5', SLEEP(5), 0) --
```

- > ¿Qué es SQL?
- > SQL Injection
- > Ataques UNION
- > Examinando la base de datos
- > Blind SQLi
- > Contramedidas para SQLi
- > Herramientas

SQLi root cause

- > La **causa madre** del problema explotado por ataques de tipo SQLi es permitir al usuario de la aplicación introducir libremente cadenas de caracteres sin control ninguno.
- > Por lo tanto la solución genérica sería evitar que se pudieran introducir caracteres especiales, sin haberlas sanitizado antes.
- > Por ejemplo, una comilla doble " debería transformarse en \" para ser interpretada como texto y no como el carácter que cierra o abre la sentencias de una consulta.
- > Estas contramedidas son dependientes del lenguaje de programación elegido para construir la aplicación.

Defensa según OWASP....

- > Según OWASP existen 4 opciones diferentes de **prevención de SQLi** :
 - **#1**: Uso de sentencias preparadas (con consultas parametrizadas).
 - **#2**: Uso de stored procedures construidos adecuadamente.
 - **#3**: Validación de entradas mediante lista blanca. **[DESACONSEJADO]**
 - **#4**: Escapar todas las entradas proporcionadas por el usuario **[FUERTEMENTE DESACONSEJADO]**

#1 Sentencias preparadas

- > Las sentencias preparadas son sencillas de escribir y fáciles de entender.
- > Estas consultas parametrizadas obligan al desarrollador a definir todo el código SQL primero y luego pasar cada parámetro a la consulta.
- > La base de datos siempre distinguirá entre código y datos , sin importar la entrada que proporcione el usuario.
- > Las sentencias preparadas aseguran que un atacante no pueda cambiar la intención de una consulta, incluso si se insertan comandos SQL por parte de un atacante.

#1 Sentencias preparadas

```
// This should REALLY be validated too
String custname = request.getParameter("customerName");
// Perform input validation to detect attacks
String query = "SELECT account_balance FROM user_data WHERE user_name = ? ";
PreparedStatement pstmt = connection.prepareStatement(query);
pstmt.setString(1, custname);
ResultSet results = pstmt.executeQuery();
```

```
String query = "SELECT account_balance FROM user_data WHERE user_name = ?";
try {
    OleDbCommand command = new OleDbCommand(query, connection);
    command.Parameters.Add(new OleDbParameter("customerName", CustomerName Name.Text));
    OleDbDataReader reader = command.ExecuteReader();
    // ...
} catch (OleDbException se) {
    // error handling
}
```

#2 Stored Procedures

- > Si bien los stored procedures no siempre están a salvo de SQLi, los desarrolladores pueden usar ciertos constructores estándares de programación para evitar inyecciones de código.
- > Este enfoque tiene el mismo efecto que el uso de consultas parametrizadas, siempre y cuando los procedimientos almacenados se implementen de forma segura.
- > Requiere que el desarrollador construya sentencias SQL con parámetros que se parametrizan automáticamente.
- > La diferencia entre las sentencias preparadas y los stored procedures es que el código SQL de uno se define y almacena en la propia base de datos, y luego se invoca desde la aplicación.

#2 Stored Procedures

```
// This should REALLY be validated
String custname = request.getParameter("customerName");
try {
    CallableStatement cs = connection.prepareCall("{call sp_getAccountBalance(?)}");
    cs.setString(1, custname);
    ResultSet results = cs.executeQuery();
    // ... result set handling
} catch (SQLException se) {
    // ... logging and error handling
}
```

```
Try
    Dim command As SqlCommand = new SqlCommand("sp_getAccountBalance", connection)
    command.CommandType = CommandType.StoredProcedure
    command.Parameters.Add(new SqlParameter("@CustomerName", CustomerName.Text))
    Dim reader As SqlDataReader = command.ExecuteReader()
    ' ...
Catch se As SQLException
    'error handling
End Try
```

- > ¿Qué es SQL?
- > SQL Injection
- > Ataques UNION
- > Examinando la base de datos
- > Blind SQLi
- > Contramedidas para SQLi
- > Herramientas

Automatic SQLi and database tool

- > [sqlmap](#) es una herramienta de código abierto que automatiza el proceso de detección y explotación de fallas de inyección de SQL y la toma de control de los servidores de bases de datos.
- > Viene con un potente motor de detección y una amplia gama de capacidades, tales como:
 - Footprint de la base de datos.
 - Obtención de datos de la base de datos.
 - Acceso al sistema de archivos.
 - Ejecución de comandos en el sistema operativo.



Dudas?

